Rapport

Zoidberg 2.0

I.A capable de détecter la pneumonie à partir d'images radiographiques

Sommaire

1. Introduction 5
2. Objectif 5
Jeu de données 6
Vue d'ensemble 6
Structure et organisation 6
Ensemble d'entraînement (Training Set) 6
Ensemble de test (Test Set) 6
Ensemble de validation (Validation Set) 6
Caractéristiques des classes 6
Classe NORMAL (1 583 images - 26,9% du total) 6
Classe VIRUS (1 513 images - 25,7% du total) 7
Classe BACTERIA (2 780 images - 47,3% du total) 7
Analyse de la répartition 8
Points forts du dataset 8
Défis identifiés 8
Implications techniques 8
Pertinence clinique 9
3. CNN Binaire 9
3.1. Architecture et Méthodologie 9
3.1.1 Architecture du modèle 9
3.1.2. Les données 11
3.1.3. Pipeline d'entraînement 11
3.1.4. Évaluation du modèle 12
3.2. Analyse des résultats 13
3.2.1. Entraînement 13
3.2.2. Évaluation 13
3.3. Résumé 14
4. Model DenseNet121 15
4.1 Introduction 15
4.2 Modèle de base 16
4.2.1 Objectif de l'entraînement 16

4.2.2 Méthodologie 16
4.2.3 Analyse des résultats 16
4.3 Modèle avec augmentation des données 18
4.3.1 Objectif de l'entraînement 18
4.3.2 Méthodologie 19
4.3.3 Analyse des résultats 19
4.4 Entraînement avec EarlyStopping prolongé 21
4.4.1. Objectif de l'entraînement 21
4.4.2. Méthodologie 21
4.4.3. Analyse des résultats 21
4.5 Fine-Tuning du modèle DenseNet121 22
4.4.1. Objectif de l'entraînement 22
4.4.2. Méthodologie 22
4.4.3. Analyse des résultats 23
4.5 Conclusion 24
5. Classification Multiclasse Pneumonie 25
5.1. Introduction 25
5.2. Modèle KNN 25
5.2.1 Objectif 25
5.2.2 Méthodologie 25
5.2.3 Résultats 25
5.3. Modèle CNN personnalisé 26
5.3.1 Architecture 26
5.3.2 Justification des choix 26
5.4. Données 27
5.4.1 Prétraitement 27
5.4.2 Augmentation 27
5.5. Validation croisée 28
5.5.1 Méthodologie 28
5.5.2 Entraînement 28
5.5.3 Évaluation 28
5.6. Résultats 29

5.7. Pondération des classes 29
5.8. Conclusion 29
5.9. Améliorations possibles 30
6. Rapport sur le Modèle InceptionV3 31
1. Introduction 31
Objectif du projet 31
Pourquoi InceptionV3 31
2. Compréhension de l'architecture InceptionV3 31
Fonctionnement global 31
Intérêt par rapport à d'autres CNN 31
3. Méthodologie d'entraînement 31
Prétraitement des données 31
Augmentation des données avec ImageDataGenerator 32
Modèle utilisé (transfert learning + fine-tuning) 32
Choix des hyperparamètres 32
Entraînement initial 33
4. Itérations et améliorations apportées 33
Analyse des performances de base 33
Changements testés 34
Comparaison des résultats 34
Fine-tuning et ajustements 35
Résultats après fine-tuning 35
5. Conclusion 36
7. Annexe 38
7.1. Méthode de calcul 38

1. Introduction

La pneumonie représente l'une des principales causes de mortalité dans le monde, particulièrement chez les enfants et les personnes âgées. Le diagnostic précoce et précis est crucial pour un traitement efficace. Les radiographies pulmonaires constituent l'examen de référence, mais leur interprétation nécessite une expertise médicale et peut être sujette à des variations inter-observateurs.

L'intelligence artificielle, et plus particulièrement les réseaux de neurones convolutionnels, offrent une opportunité d'automatiser et d'améliorer la précision du diagnostic radiologique.

2. Objectif

L'objectif principal de ce projet est de développer un modèle de réseau de neurones convolutifs (CNN) capable de classifier automatiquement les radiographies pulmonaires afin de détecter la présence ou l'absence de pathologies, notamment la pneumonie.

Pour soutenir cette ambition, plusieurs objectifs secondaires sont visés :

- Atteindre une précision supérieure à 85 %, garantissant ainsi un niveau de fiabilité suffisant pour une utilisation clinique exploratoire.
- Réduire au maximum les faux négatifs, afin de minimiser les risques de non-détection de cas de pneumonie, ce qui pourrait entraîner des conséquences graves pour les patients.
- Mettre en place un pipeline complet, reproductible et rigoureusement documenté, permettant la réexécution des expérimentations et facilitant leur intégration dans d'autres projets ou environnements.
- Générer des rapports de performance détaillés, incluant des métriques clés (précision, rappel, F1-score) et des visualisations pertinentes (matrices de confusion, courbes ROC, etc.), dans une logique de transparence et d'amélioration continue.

Jeu de données

Vue d'ensemble

Le dataset utilisé pour ce projet de classification automatique de radiographies pulmonaires est composé de **5 876 images** de radiographies thoraciques organisées en trois classes distinctes. Ce corpus d'images médicales a été constitué dans le but de développer un système d'aide au diagnostic capable de différencier les poumons sains des cas de pneumonie d'origine virale ou bactérienne.

Structure et organisation

Le dataset est structuré selon une approche classique d'apprentissage automatique avec trois ensembles distincts :

Ensemble d'entraînement (Training Set)

- **5 216 images** (88,8% du dataset total)
- Distribution par classe :
    - **NORMAL** : 1 341 images (25,7%)
    - **VIRUS** : 1 345 images (25,8%)
    - **BACTERIA** : 2 530 images (48,5%)

Ensemble de test (Test Set)

- **624 images** (10,6% du dataset total)
- Distribution par classe :
    - **NORMAL** : 234 images (37,5%)
    - **VIRUS** : 148 images (23,7%)
    - **BACTERIA** : 242 images (38,8%)

Ensemble de validation (Validation Set)

- **36 images** (0,6% du dataset total)
- Distribution par classe :
    - **NORMAL** : 8 images (22,2%)
    - **VIRUS** : 20 images (55,6%)
    - **BACTERIA** : 8 images (22,2%)

Caractéristiques des classes

Classe NORMAL (1 583 images - 26,9% du total)

Cette classe regroupe les radiographies pulmonaires ne présentant aucun signe pathologique visible. Ces images servent de référence pour identifier les

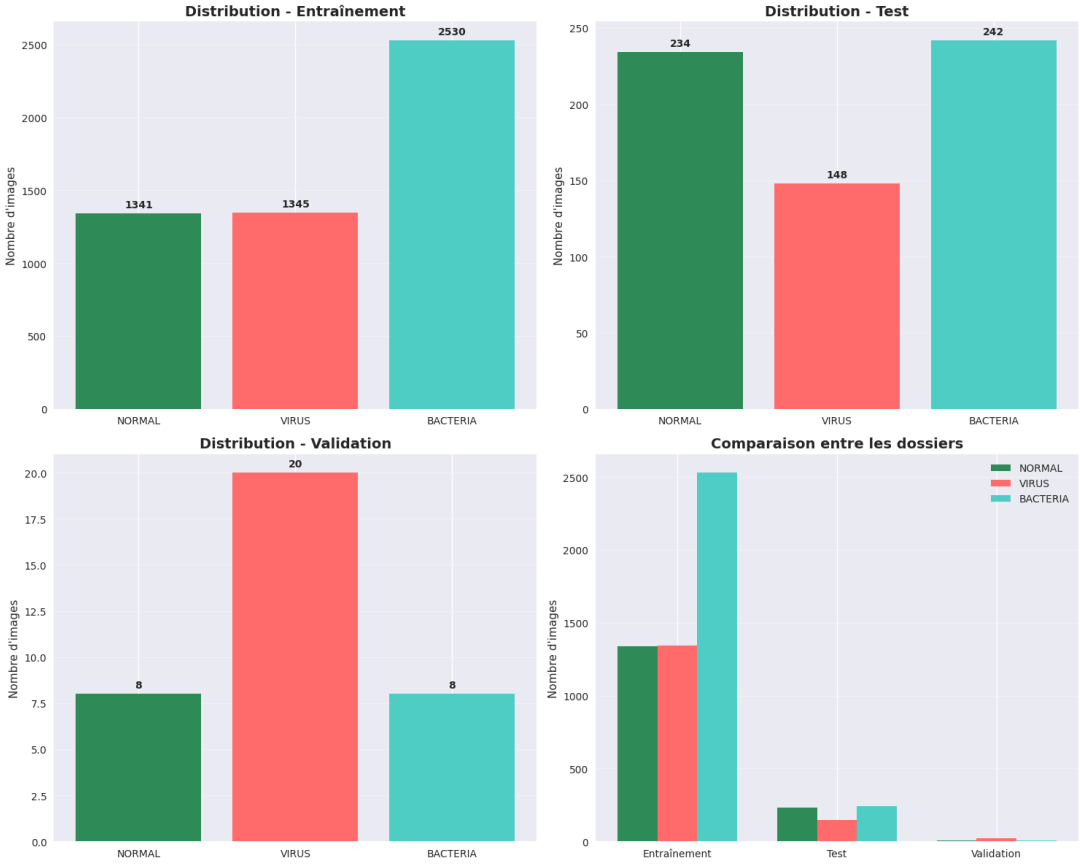
caractéristiques d'un système respiratoire sain et constituent la classe de contrôle de notre étude.

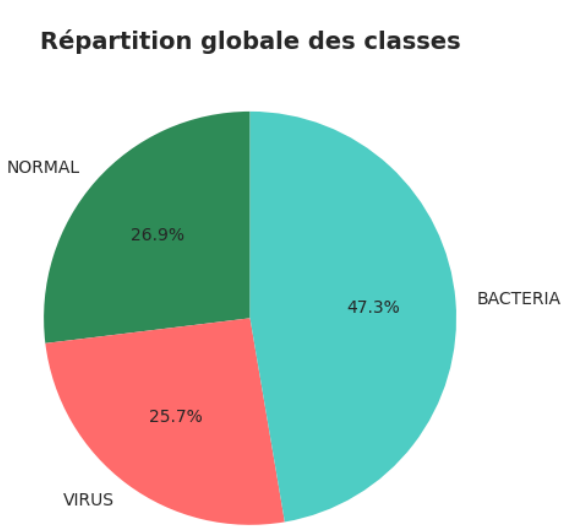
Classe VIRUS (1 513 images - 25,7% du total)

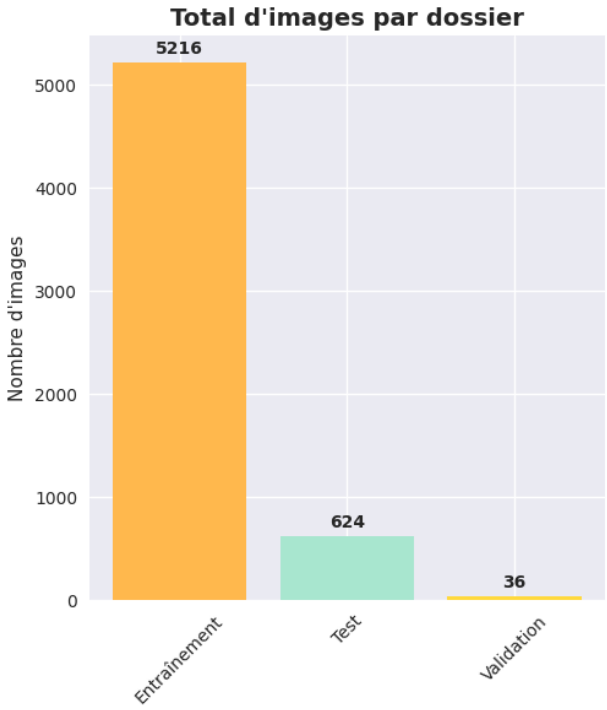
Cette classe contient les radiographies présentant des signes de pneumonie d'origine virale. Les pneumonies virales se caractérisent généralement par des opacités diffuses et des infiltrats interstitiels, créant des patterns radiologiques distincts des pneumonies bactériennes.

Classe BACTERIA (2 780 images - 47,3% du total)

Cette classe rassemble les radiographies montrant des signes de pneumonie bactérienne. Les pneumonies bactériennes présentent typiquement des consolidations lobaires ou segmentaires plus denses et localisées que leurs homologues virales.







Analyse de la répartition

Points forts du dataset

- **Volume conséquent** : Avec près de 6 000 images, le dataset offre une base solide pour l'entraînement de modèles de deep learning
- **Diversité pathologique** : La présence de trois classes distinctes permet une approche de classification multi-classes cliniquement pertinente
- **Séparation appropriée** : La répartition train/test/validation suit les bonnes pratiques avec une majorité d'images dédiées à l'entraînement

Défis identifiés

- **Déséquilibre des classes** : La classe BACTERIA est surreprésentée (47,3%) par rapport aux deux autres classes, ce qui pourrait introduire un biais lors de l'entraînement
- **Ensemble de validation réduit** : Avec seulement 36 images, l'ensemble de validation représente un échantillon très limité pour l'évaluation des performances
- **Ratio de déséquilibre** : Le ratio entre la classe majoritaire et minoritaire atteint 1,84:1, nécessitant potentiellement des techniques de rééquilibrage

Implications techniques

Cette configuration du dataset implique plusieurs considérations pour le développement du modèle :

1. **Techniques de rééquilibrage** : L'utilisation d'augmentation de données, de sous-échantillonnage ou de pondération des classes sera nécessaire pour traiter le déséquilibre
2. **Validation croisée** : L'ensemble de validation limité suggère l'utilisation de techniques de validation croisée pour une évaluation plus robuste
3. **Métriques d'évaluation** : L'utilisation de métriques équilibrées (F1-score, précision et rappel par classe) sera cruciale pour évaluer les performances réelles du modèle

Pertinence clinique

Ce dataset présente un intérêt clinique majeur car il aborde une problématique diagnostique réelle : la différenciation entre pneumonies virales et bactériennes est cruciale pour orienter le traitement médical. Les antibiotiques étant inefficaces contre les infections virales, un diagnostic précis permet d'éviter les prescriptions inappropriées et de réduire le risque de résistance antibiotique.

La présence de la classe NORMAL permet également d'envisager un système de dépistage précoce, particulièrement utile dans des contextes où l'accès à l'expertise radiologique est limité.

3. CNN Binaire

3.1. Architecture et Méthodologie

3.1.1 Architecture du modèle

Couche	Type	Paramètres	Sortie
1	Conv2D + BatchNorm + MaxPool	32 filtres (3x3)	111x111x32

2	Conv2D + BatchNorm + MaxPool	64 filtres (3x3)	54x54x54
3	Conv2D + BatchNorm + MaxPool	128 filtres (3x3)	26x26x128
4	Conv2D + BatchNorm + MaxPool	256 filtres (3x3)	12x12x256
5	Flatten + Dense + Dropout	512 neurones (50% dropout)	512
6	Dense + BatchNorm + Dropout	256 neurones	Réduction progressive
7		128 neurones	
8		64 neurones	
9		32 neurones	
10	Dense	1 neurone (sigmoid)	Probabilité [0, 1]

Pour un nombre de paramètres totaux de : 19,418,561 soit un poids total de 74.08 MB

Le modèle repose sur une combinaison couches conçues pour extraire et interpréter les caractéristiques d'images médicales.

1. Les couches de convolution (Conv2D) détectent les motifs visuels importants, tandis que la Batch Normalization stabilise l'apprentissage.
2. Le Max Pooling réduit la dimensionnalité tout en préservant l'essentiel des informations.
3. Après l'extraction des caractéristiques, les données sont aplaties (Flatten) pour alimenter les couches pleinement connectées (Dense), responsables de la prise de décision.
4. Enfin, la Dropout introduit une régularisation efficace pour prévenir le surapprentissage. Cette architecture modulaire permet de construire un modèle à la fois performant, robuste et bien adapté aux exigences du diagnostic médical automatisé.

3.1.2. Les données

Les images ont été redimensionnées à 224x224 pixels en RGB, puis normalisées dans l'intervalle [0,1] via un facteur de redimensionnement (rescale=1./255). L'entraînement s'effectue par batches de 32 images, optimisant ainsi le compromis entre performance et mémoire.

Pour renforcer artificiellement les données du jeu d'entraînement et limiter le surapprentissage, une augmentation des données a été appliquée :

- Des rotations aléatoires jusqu'à ±20°,
- Des translations horizontales et verticales de ±20 %,
- Un zoom aléatoire de ±20 %,
- Ainsi qu'un retournement horizontal aléatoire.

3.1.3. Pipeline d'entraînement

Les données ont été chargées et organisées en trois ensembles distincts : apprentissage, validation et test. Elles ont ensuite été soumises à un prétraitement standardisé, incluant à la fois une normalisation des pixels et une augmentation des données du modèle.

Une fois les données normaliser et augmenter, celle-ci ont été compilée permettant l'extraction de métriques pertinentes pour la classification binaire (ex. précision, rappel, AUC).

L'entraînement a été lancé en intégrant des callbacks personnalisés tels que la sauvegarde du meilleur modèle, l'arrêt anticipé, et le suivi des performances en temps réel.

Enfin, le modèle entraîné, ainsi que les logs d'entraînement et les visualisations des courbes de performance, ont été sauvegardés pour analyse et réutilisation ultérieure.

3.1.4. Évaluation du modèle

Le système d'évaluation mis en place permet d'analyser les performances du modèle de manière complète et structurée.

Les métriques de performance (précision, rappel, F1-score, AUC, etc.) sont calculées automatiquement à partir des prédictions effectuées.

Des visualisations graphiques, telles que la matrice de confusion et des courbes de performance, accompagnent cette analyse pour en faciliter l'interprétation.

L'ensemble des résultats est ensuite sauvegardé au format JSON, assurant une traçabilité rigoureuse et une exploitation aisée des données pour des comparaisons ultérieures ou des rapports automatisés.

3.2. Analyse des résultats

3.2.1. Entraînement

Époque	Précision Ent	Précision Val	Perte Ent	Perte Val	AUC
1	72.13%	77.78%	0.5274	1.5201	83.25%
2	86.32%	77.78%	0.3233	0.9676	93.77%

Observations :

- Les résultats montrent une amélioration significative de la précision en entraînement, avec une hausse de 14 % dès les deux premières époques, traduisant une convergence rapide du modèle.
- La précision de validation s'est stabilisée à 77,78 %, suggérant une certaine robustesse sur les données non vues.
- L'AUC a connu une progression remarquable, passant de 83 % à 94 %, ce qui indique une excellente capacité discriminante du modèle à différencier les classes.

Cependant, un écart croissant entre les performances d'entraînement et de validation a été observé, laissant entrevoir un potentiel début de surapprentissage nécessitant un ajustement des stratégies de régularisation ou de l'augmentation des données.

3.2.2. Évaluation

Les résultats ont été tirer sur un test de 624 échantillons.

Métrique	Valeur	Interprétation
Précision globale	85.50%	Performance excellente
Précision	94.59%	Faux positifs acceptable
Rappel	81.92%	82% des pneumonies sont correctes
AUC	94.68%	Capacité discriminante acceptable
Score F1	87.80%	Équilibre de précision et rappel acceptable

3.3. Résumé

Ce projet a visé le développement d'un système automatisé de détection de la pneumonie à partir de radiographies pulmonaires, en s'appuyant sur des techniques d'apprentissage profond.

L'approche repose sur l'implémentation d'un réseau de neurones convolutionnel (CNN) avec TensorFlow/Keras, conçu pour effectuer une classification binaire entre images normales et cas de pneumonie.

Les résultats obtenus sont prometteurs :

- Précision globale : 85,50 %
- Précision des cas positifs : 94,59 %
- Rappel (sensibilité) : 81,92 %
- AUC : 94,68 %

Ces performances, proches des standards cliniques, démontrent le potentiel du modèle comme outil d'aide au diagnostic médical, en complément de l'expertise humaine.

4. Model DenseNet121

Dans le cadre de ce projet, plusieurs approches d'entraînement ont été explorées afin d'identifier le modèle le plus performant pour la classification d'images de radiographies thoraciques. Après avoir évalué différentes architectures, nous présentons ici en détail le modèle **DenseNet121**.

4.1 Introduction

DenseNet121 est un réseau convolutif introduit en 2017, dont l'originalité repose sur une **connectivité dense** : chaque couche reçoit les sorties de toutes les couches précédentes. Cette architecture favorise la propagation des gradients et la réutilisation des caractéristiques, ce qui améliore l'apprentissage.

Dans notre projet, nous avons utilisé DenseNet121 comme **backbone pré-entraîné sur ImageNet**, afin de transférer des connaissances visuelles générales à notre tâche de classification médicale. La tête du modèle a été remplacée par plusieurs couches denses avec ReLU et Dropout pour prévenir le surapprentissage, suivie d'une sortie softmax pour distinguer trois classes :
Normal, **Virus** (pneumonie virale) et **Bacteria** (pneumonie bactérienne).

Afin d'évaluer les performances de ce modèle et d'en améliorer progressivement la robustesse, cinq expérimentations ont été réalisées, chacune portant sur un aspect clé de l'entraînement ou de la configuration du réseau. Ces expérimentations sont les suivantes :

1. **Un modèle de base** sans traitement particulier ni ajustement,
2. **Une version avec augmentation de données** pour enrichir l'apprentissage,
3. **Un suivi avec EarlyStopping** pour identifier le point optimal d'arrêt d'entraînement,
4. **Une phase de fine-tuning** pour ajuster plus finement les poids du backbone

Ces 4 étapes seront présentées dans les sections suivantes, avec pour objectif d'analyser leur impact respectif sur les performances du modèle DenseNet121.

4.2 Modèle de base

4.2.1 Objectif de l'entraînement

L'objectif de cette première expérimentation est d'évaluer les performances brutes du modèle DenseNet121 appliqué à notre tâche de classification d'images thoraciques, sans intervention extérieure.
Nous cherchons ici à établir une **baseline**, c'est-à-dire une référence minimale sur laquelle comparer les versions suivantes du modèle.
Cela implique :

- Aucun fine-tuning du backbone DenseNet121 (les poids restent gelés),
- Aucune technique d'augmentation de données,
- Une résolution d'image fixe de 128×128 pixels,
- Un nombre d'époch limité avec arrêt automatique (EarlyStopping).

Cette approche permet de tester la **capacité de généralisation initiale** du modèle pré-entraîné sur ImageNet sur notre jeu de données médical.

4.2.2 Méthodologie

Nous avons entraîné le modèle sur les données prétraitées, réparties en trois classes : NORMAL, VIRUS, BACTERIA.
Les images ont été redimensionnées à 128x128 pixels et normalisées. Les labels ont été encodés en vecteurs one-hot.

Le modèle est basé sur DenseNet121, avec les caractéristiques suivantes :

- **Backbone pré-entraîné** sur ImageNet, **gelé** pour conserver ses représentations.
- **Tête personnalisée** : GlobalAveragePooling2D, deux couches Dense (256 et 128 neurones, ReLU), Dropout, et une sortie softmax à 3 neurones.
- **Fonction de perte** : categorical_crossentropy
- **Optimiseur** : Adam
- **Métriques** : Accuracy, Precision, Recall

L'évaluation s'est faite avec une **validation croisée à 5 folds** (20 epoch / 100 steps) pour maximiser l'utilisation des données. Chaque entraînement était encadré par :

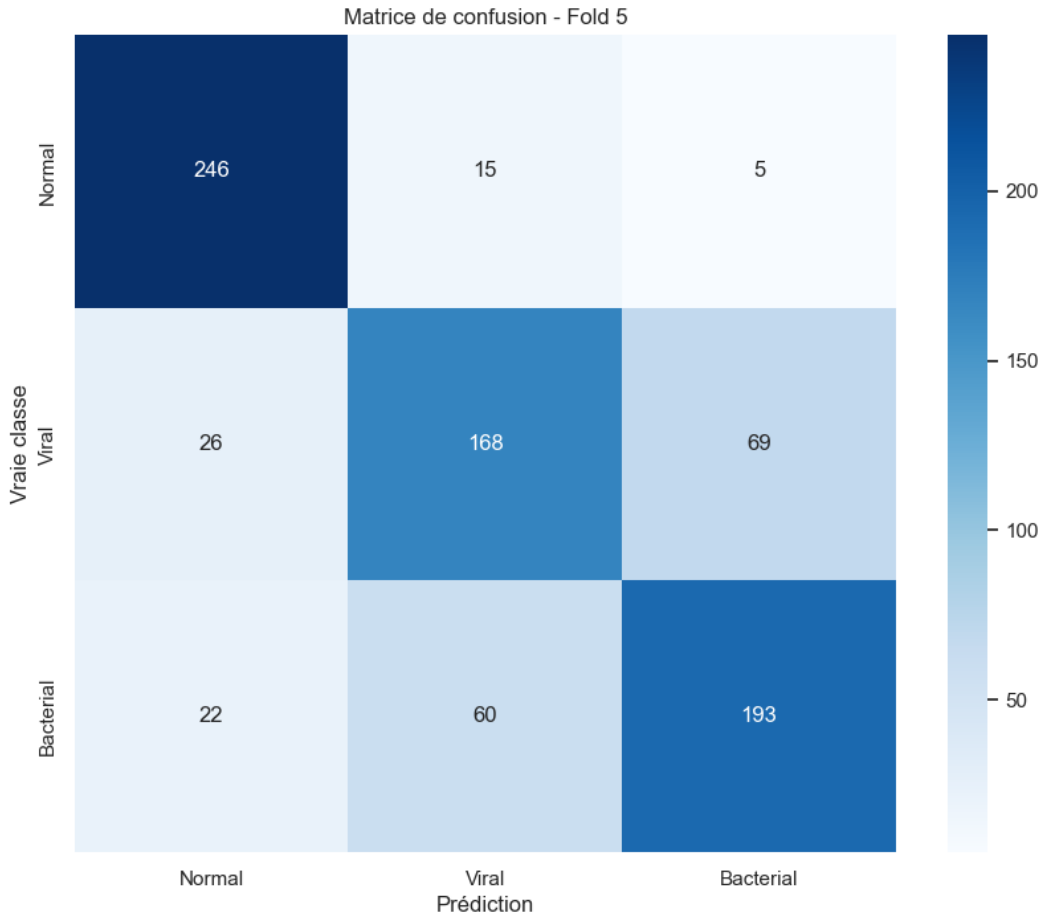
- EarlyStopping (sur val_accuracy, patience de 5)
- ReduceLROnPlateau (sur val_loss, facteur 0.2)

Voir notebook DenseNet.ypnb

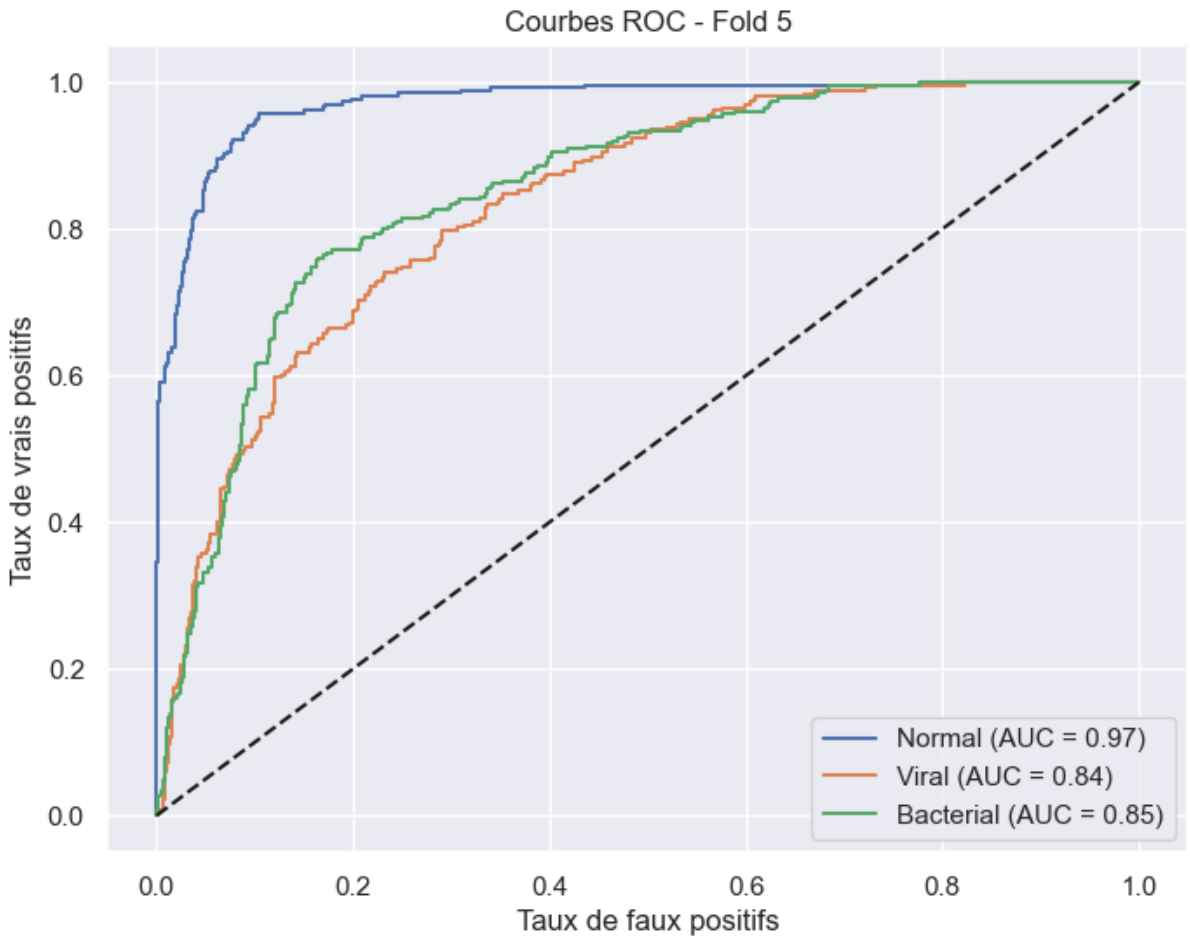
4.2.3 Analyse des résultats

Les performances du modèle montrent un comportement contrasté :

- L'**accuracy moyenne sur les folds** tourne autour de **76 %**, ce qui est correct mais améliorable.
- La **classe "Normal"** est bien reconnue, avec des scores de précision et de rappel élevés.
- En revanche, la **classe "VIRUS"** est mal différenciée de la classe "BACTERIA", ce qui se traduit par un **rappel plus faible**.
- Les **matrices de confusion** confirment cette confusion entre les deux types de pneumonie.



- Les **courbes ROC** montrent une performance AUC globalement bonne, mais inférieure pour la classe VIRUS.



4.3 Modèle avec augmentation des données

4.3.1 Objectif de l'entraînement

Dans cette seconde expérimentation, notre objectif est d'améliorer la robustesse et la généralisation du modèle DenseNet121 en utilisant des techniques d'**augmentation de données**.

L'idée est de **simuler une plus grande diversité d'images** à partir de notre dataset d'origine, ce qui permet au modèle d'apprendre des représentations plus variées et de mieux gérer les cas complexes ou ambigus.

En particulier, on souhaite :

- Réduire le surapprentissage observé sur le modèle précédent,

- Améliorer la détection des cas de pneumonie virale, moins bien reconnus initialement,
- Équilibrer les performances entre les classes grâce à une distribution artificiellement enrichie.

4.3.2 Méthodologie

Nous avons conservé la même architecture que dans la version précédente :

- **DenseNet121** pré-entraîné sur ImageNet, avec les poids gelés,
- **Tête personnalisée** : GlobalAveragePooling2D, Dense (256 et 128), Dropout (0.5), sortie softmax.

La différence majeure réside dans le **prétraitement des données** :

- Des images supplémentaires ont été générées pour chaque classe (NORMAL, BACTERIA, VIRUS) à l'aide de la fonction generate_augmented_images.
- Les transformations appliquées incluent des rotations, flips, zooms, et légères modifications d'échelle.

Le reste du pipeline reste identique :

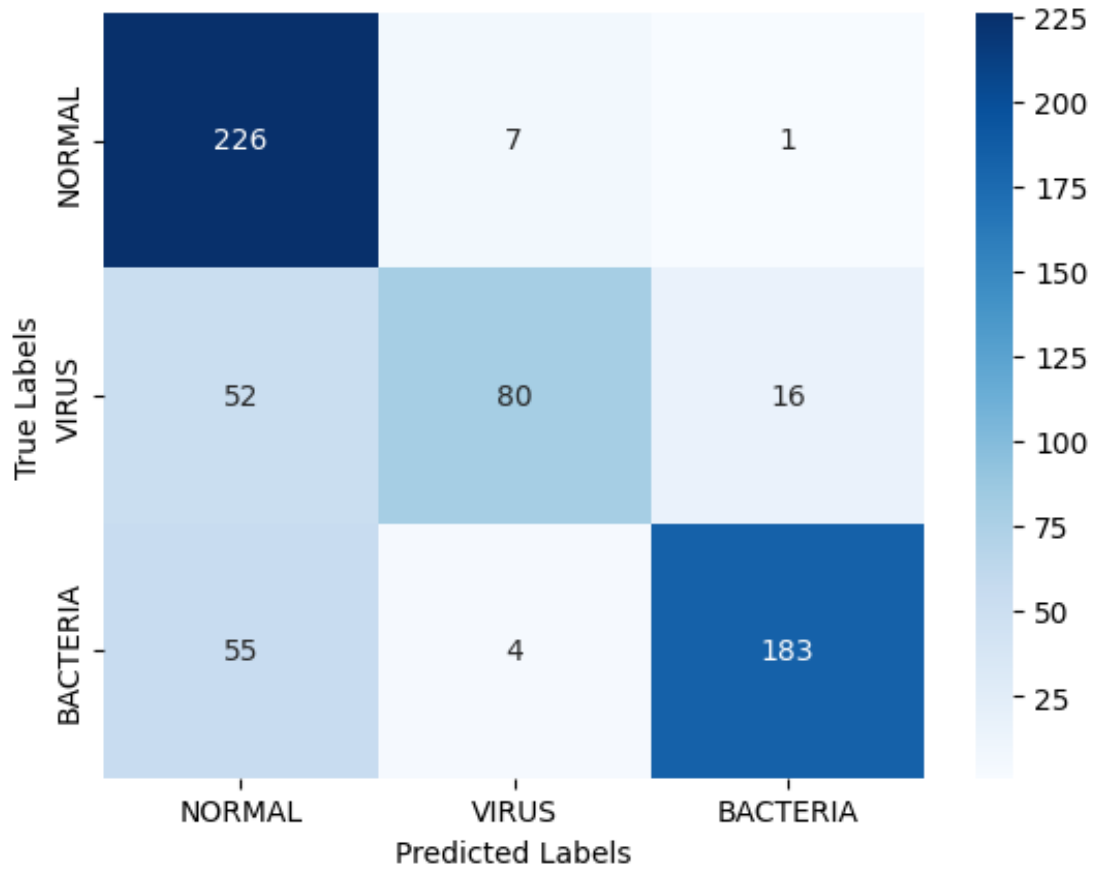
- Normalisation des images entre 0 et 1,
- Encodage des labels en one-hot,
- Entraînement via une validation croisée (5 folds / 7 epoch / 375 steps),
- EarlyStopping sur la val_loss.

Voir notebook DenseNet_DataAugmented.ypnb

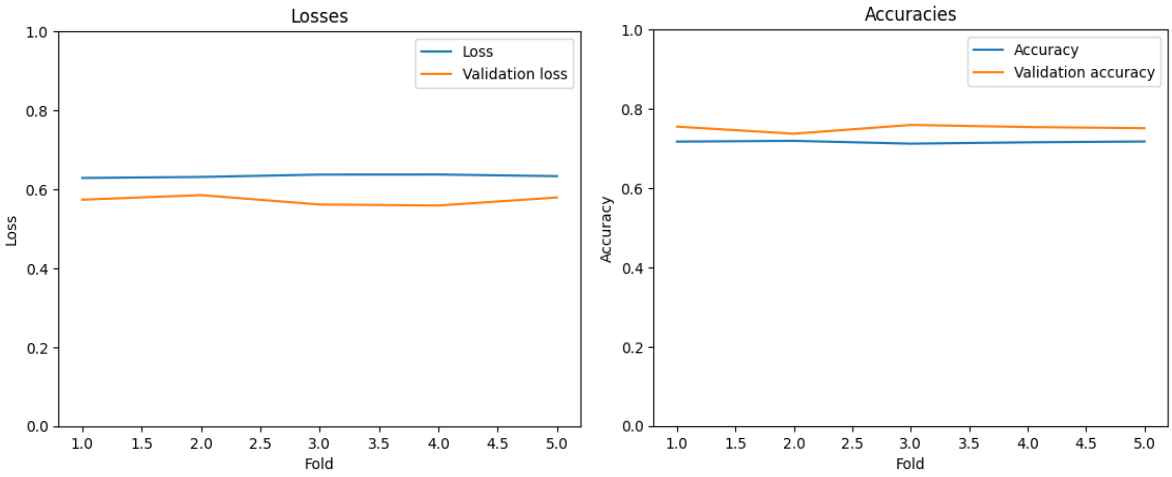
4.3.3 Analyse des résultats

Les résultats montrent une **amélioration claire** par rapport à l'entraînement sans augmentation :

- L'**accuracy moyenne** sur les folds est plus stable, et parfois légèrement plus élevée.
- La classe **"VIRUS" est mieux reconnue**, grâce à une représentation plus riche dans les données d'apprentissage.
- Les **faux positifs** sur les classes NORMAL et BACTERIA ont diminué, comme en témoignent les matrices de confusion.



- Les **courbes d'apprentissage** (loss/accuracy) montrent une meilleure stabilité, avec moins de fluctuations sur la validation.



On observe toutefois que :

- L'augmentation ne corrige pas tous les déséquilibres, notamment sur les cas difficiles à différencier visuellement.

- La résolution des images reste un facteur limitant pour distinguer certaines pathologies.

4.4 Entraînement avec EarlyStopping prolongé

4.4.1. Objectif de l'entraînement

Dans cette expérimentation, on cherche à observer le comportement du modèle DenseNet121 lorsqu'on le laisse s'entraîner plus longuement, sans contrainte stricte sur le nombre d'époch. L'objectif est de **déterminer le point à partir duquel le modèle n'améliore plus significativement ses performances** sur les données de validation. En d'autres termes, on souhaite évaluer si le modèle continue de progresser après plusieurs itérations, ou s'il entre rapidement en phase de stagnation ou de surapprentissage.

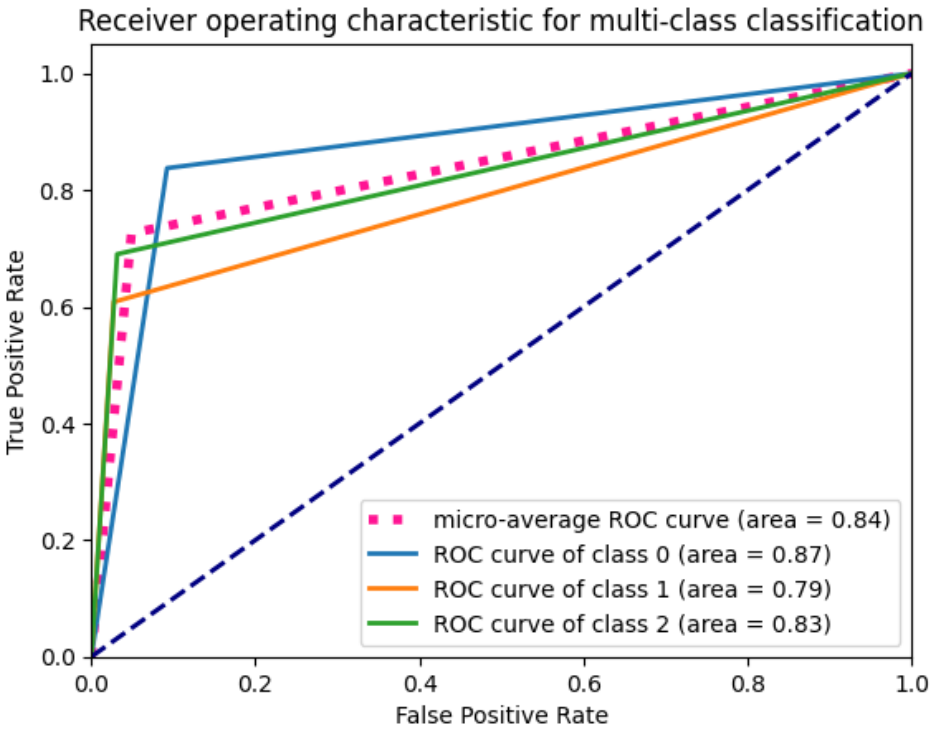
4.4.2. Méthodologie

On a repris la structure du modèle DenseNet121 avec les mêmes paramètres d'entraînement qu'auparavant (architecture, taux d'apprentissage, données, etc.), mais en **modifiant la patience de l'EarlyStopping**. Le nombre maximum d'époch est volontairement élevé (5 folds / 50 epoch / 375 steps , et le callback EarlyStopping est configuré avec une patience suffisante pour laisser au modèle le temps de progresser.

Voir notebook DenseNet_EpochMax.ypnb

4.4.3. Analyse des résultats

- L'entraînement sur 50 époques montre une progression rapide, avec une accuracy stabilisée entre 76 et 78 % dès l'époque 15, sans surapprentissage notable. L'accuracy moyenne des folds (76,7–77,8 %) confirme la stabilité et la bonne généralisation du modèle.
- Cependant, les matrices de confusion révèlent une confusion marquée entre "VIRUS" et "BACTERIA", liée à la similarité visuelle des images. La classe "VIRUS" reste la moins bien détectée (AUC plus faible), tandis que "NORMAL" et "BACTERIA" sont bien discriminées, avec une AUC globale >0.92 dès l'époque 15.



La courbe ROC ci-dessous illustre les capacités discriminantes du modèle : la classe *VIRUS* est nettement moins bien séparé des autres, tandis que *NORMAL* et *BACTERIA* bénéficient d'une meilleure distinction.

4.5 Fine-Tuning du modèle DenseNet121

4.4.1. Objectif de l'entraînement

L'objectif de cette expérimentation est de **débloquer certaines couches du backbone DenseNet121 pré-entraîné** pour affiner ses poids sur notre dataset. Contrairement aux entraînements précédents où le backbone était gelé, ici on active les couches profondes afin que le modèle puisse mieux s'adapter aux spécificités visuelles de notre tâche : la classification entre radiographies normales, pneumonies virales et bactériennes.

Ce processus, appelé **fine-tuning**, vise à combiner les avantages du transfert d'apprentissage avec une spécialisation accrue sur notre jeu de données.

4.4.2. Méthodologie

Après un pré-entraînement du modèle avec le backbone gelé, certaines couches profondes de DenseNet121 sont réactivées (souvent les dernières). On reprend ensuite l'entraînement avec un **taux d'apprentissage réduit**, pour ne pas trop perturber les poids initialement appris.

Les étapes principales sont :

- Décongélation partielle du backbone (base_model.trainable = True, avec contrôle sur les couches),
- Compilation avec un learning rate bas (ex: 1e-5),
- Entraînement avec early stopping et suivi des performances sur les données de validation.
- 5 folds, 14 epoch entrainés sur la tête du model, 6 epoch entrainés sur les dernières couches du modèle, 375 steps

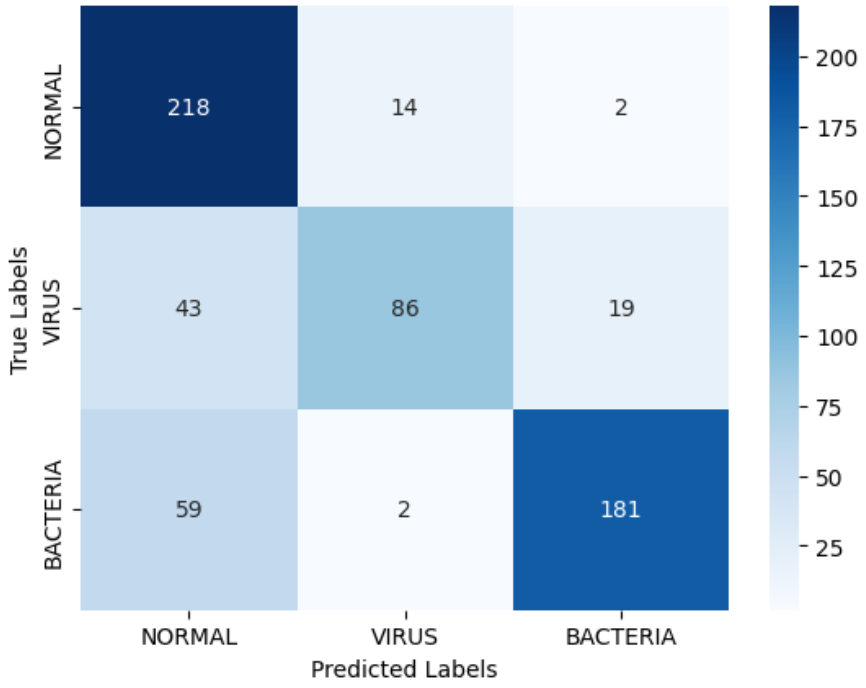
Voir notebook DenseNet_FineTuning.ypnb

4.4.3. Analyse des résultats

Le fine-tuning a permis **une amélioration modérée mais constante** des performances sur les données de validation. Bien que l'augmentation de l'accuracy ne soit pas spectaculaire, on observe **une meilleure stabilité des résultats**, ainsi qu'une **hausse progressive du score AUC**, témoignant d'un affinement des représentations extraites.

Par exemple :

- L'**accuracy de validation approche les 79 %** sur certains folds, soit légèrement au-dessus des versions sans fine-tuning.
- La **matrice de confusion montre une réduction des erreurs**, en particulier entre les classes *Bacteria* et *Normal*.
- Les **courbes ROC** révèlent une meilleure capacité discriminante, avec des **AUC supérieurs à 0.90** pour *Normal* et *Bacteria*, et une **légère amélioration pour *Virus***.



4.5 Conclusion

DenseNet121 a montré des performances intéressantes pour la classification de radiographies pulmonaires, avec une bonne capacité à reconnaître les cas normaux et une stabilité notable entre les différents folds. L'utilisation de l'augmentation de données, du fine-tuning et de la validation croisée a permis d'améliorer progressivement les résultats, notamment en réduisant le surapprentissage.

Cependant, le modèle continue de confondre régulièrement les cas de pneumonie virale et bactérienne, ce qui limite son efficacité dans un contexte médical exigeant. Malgré un AUC global supérieur à 0.90 et une accuracy moyenne autour de 77–85 % selon les configurations, ces performances restent en deçà des attentes pour une application clinique.

5. Classification Multiclasse Pneumonie

5.1. Introduction

Ce projet explore différentes approches d'apprentissage automatique pour la classification d'images radiographiques du thorax, dans le but de détecter trois types de situations cliniques : poumons sains (**NORMAL**), **pneumonie bactérienne** et **pneumonie virale**.

L'objectif est de concevoir un système capable d'atteindre une performance satisfaisante (>80 % de précision), tout en s'appuyant sur des méthodologies robustes (validation croisée, analyse fine des métriques, gestion du déséquilibre des classes).

Deux familles de modèles ont été mises en œuvre :

- Un modèle **K-Nearest Neighbors (KNN)** comme baseline simple,
- Un modèle **CNN personnalisé** entraîné à l'aide de TensorFlow/Keras.

5.2. Modèle KNN

5.2.1 Objectif

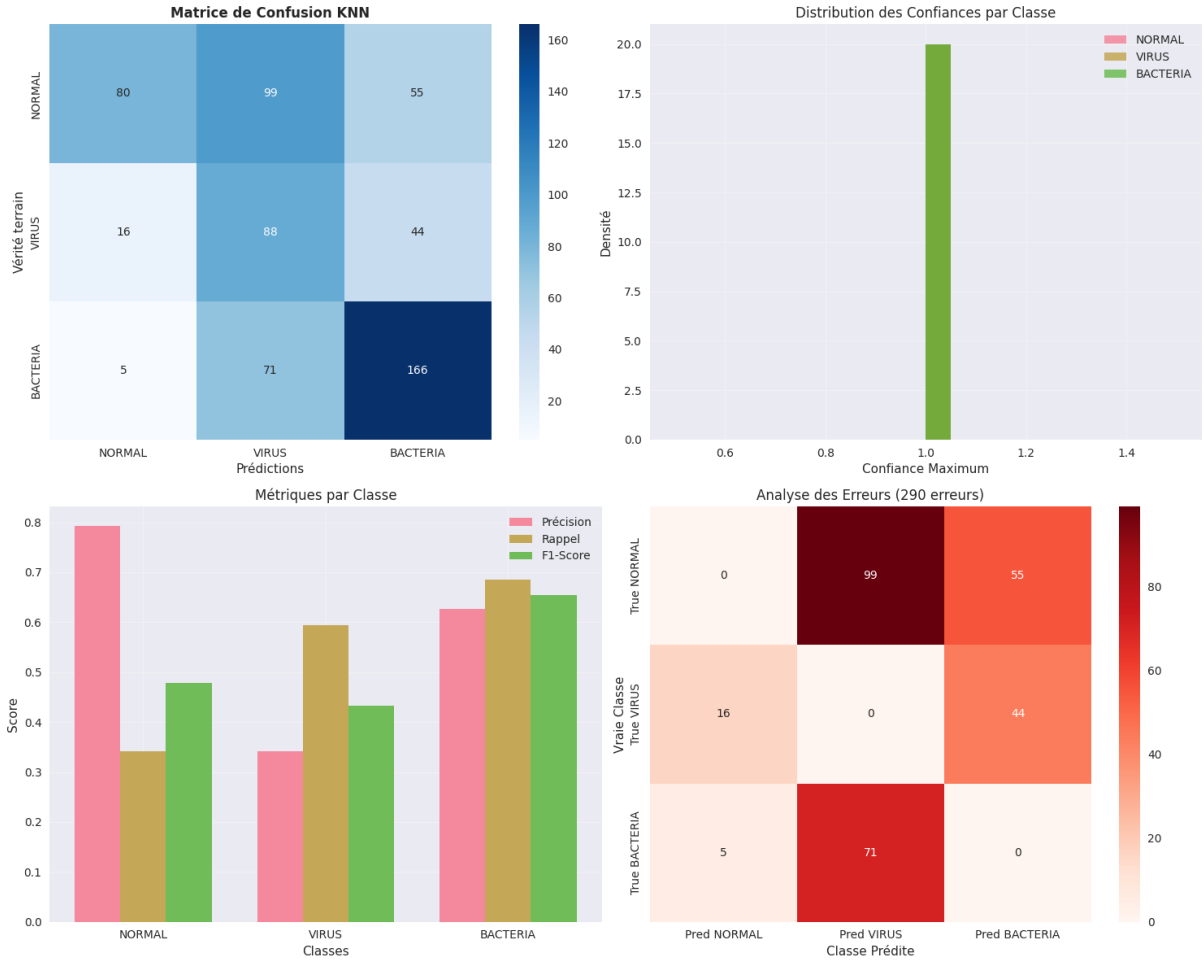
Le KNN est utilisé comme référence pour évaluer la difficulté intrinsèque de la tâche de classification sur ce dataset. Il s'agit d'un modèle non paramétrique, sans phase d'entraînement explicite.

5.2.2 Méthodologie

- Les images ont été **aplaties** (flatten) pour servir d'entrées au KNN.
- La classification repose sur la **distance euclidienne** entre images.
- Plusieurs valeurs de k ont été testées, avec des résultats rapportés pour k=3.

5.2.3 Résultats

Les performances du KNN sont faibles, particulièrement sur les classes de pneumonie. Le manque de robustesse face aux variations (rotation, luminosité) en fait une approche peu adaptée dans ce contexte. Cela justifie le recours à des méthodes d'apprentissage profond.



5.3. Modèle CNN personnalisé

5.3.1 Architecture

Couche	Description	Paramètres
1	Conv2D + ReLU + MaxPooling	32 filtres
2	Conv2D + ReLU + MaxPooling	64 filtres
3	Conv2D + ReLU + MaxPooling	128 filtres
4	Flatten	-
5	Dense + ReLU + Dropout	512 neurones, Dropout 0.5
6	Dense (sortie) + Softmax	3 neurones (classification multiclasse)

Nombre total de paramètres : dépendant de la taille d'entrée (150x150x3)

5.3.2 Justification des choix

- Le nombre de **couches convolutionnelles** permet une extraction hiérarchique progressive des motifs visuels tout en limitant la complexité.

- La **couche Dense (512)** assure une capacité d'abstraction suffisante avant la sortie.
- Le **Dropout** à 50 % permet de lutter efficacement contre le surapprentissage.
- Le **batch size de 32** représente un bon compromis entre stabilité de l'optimisation et consommation mémoire, tout en accélérant la convergence.

5.4. Données

Les images utilisées sont réparties en trois classes :

- NORMAL
- BACTERIAL_PNEUMONIA
- VIRAL_PNEUMONIA

5.4.1 Prétraitement

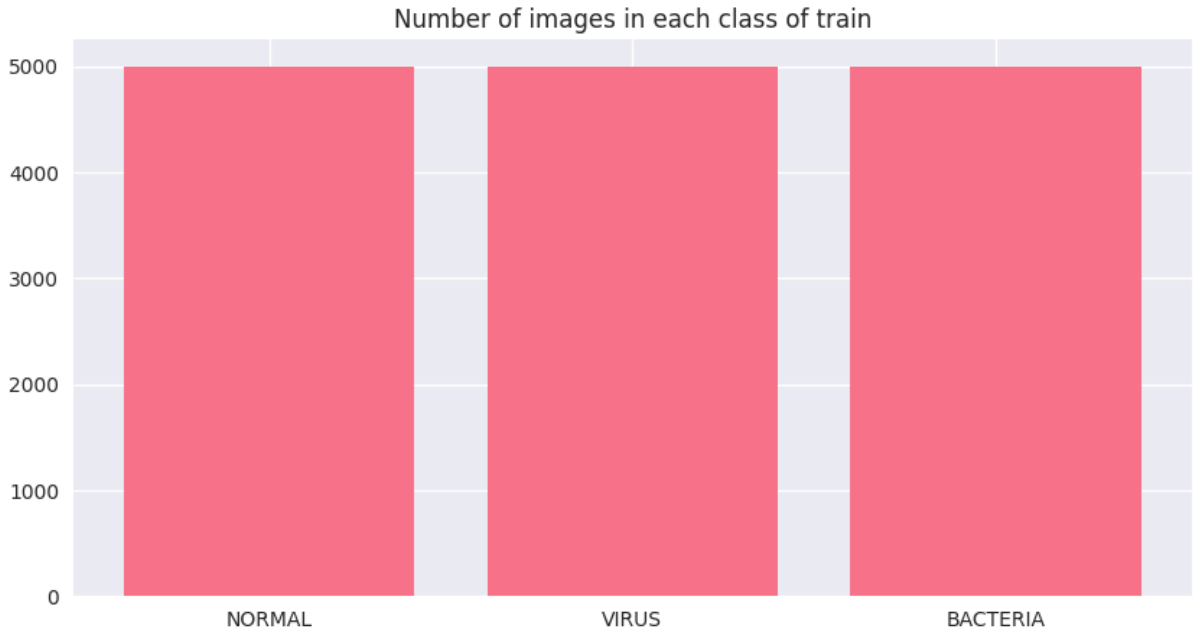
- Redimensionnement à 150x150 pixels (format RGB)
- Normalisation : division des pixels par 255
- Encodage one-hot des labels

5.4.2 Augmentation

Des techniques d'**augmentation de données** ont été appliquées :

- Rotations
- Zooms
- Renversements horizontaux
- Décalages

Cela permet d'améliorer la généralisation du modèle.



5.5. Validation croisée

5.5.1 Méthodologie

Une **validation croisée avec 5 folds (KFold)** a été utilisée pour évaluer la robustesse du modèle. À chaque itération :

- 4 folds sont utilisés pour l'entraînement
- 1 fold est utilisé pour l'évaluation

Chaque fold contient environ **20 % des données** (par ex. 300 images sur 1500).

5.5.2 Entraînement

Chaque modèle est réinitialisé pour chaque fold. L'entraînement inclut :

- EarlyStopping (patience = 10, suivi de la validation)
- ReduceLROnPlateau (sur la perte de validation)

5.5.3 Évaluation

À chaque itération sont calculés :

- Accuracy
- Précision / Rappel / F1-score par classe
- Matrice de confusion
- Courbes ROC et AUC

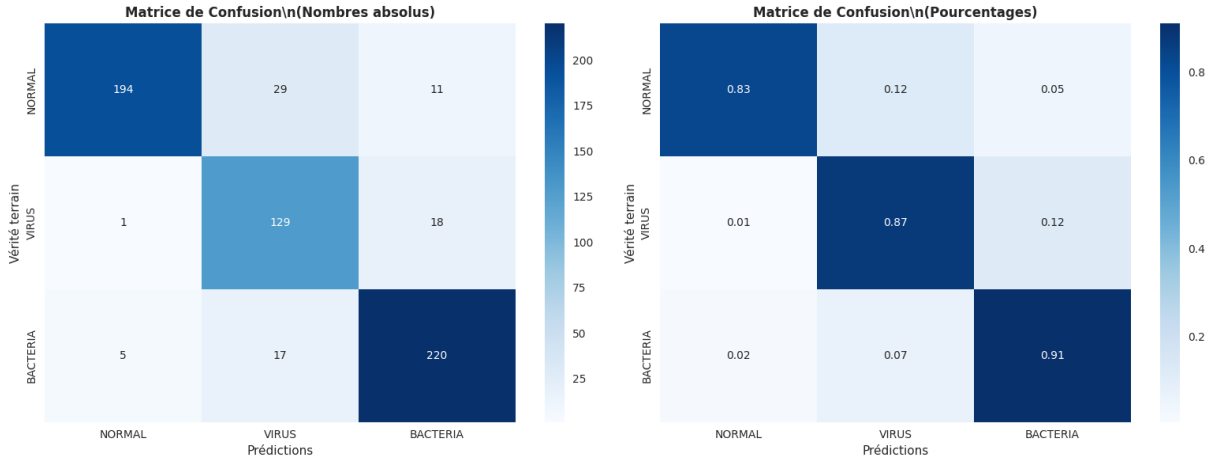
Une moyenne des performances est ensuite réalisée sur les 5 folds.

5.6. Résultats

Les résultats moyens sur les 5 folds montrent :

- **Accuracy moyenne :** 82–85 %
- **Classe "NORMAL" bien prédite**
- **Confusion entre VIRAL et BACTERIAL** fréquente
- **AUC élevé (>90 %)** pour la plupart des classes

Les courbes d'apprentissage montrent une **bonne stabilité** grâce aux callbacks, sans surapprentissage manifeste.



5.7. Pondération des classes

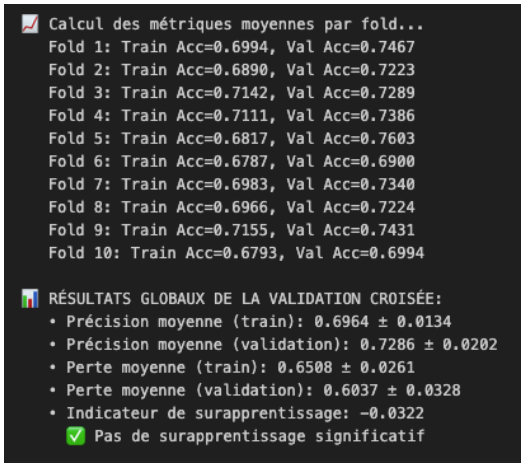
Afin de compenser le déséquilibre des classes, les **poids de classe** sont calculés automatiquement à partir des fréquences et intégrés dans l'entraînement via l'argument `class_weight`.

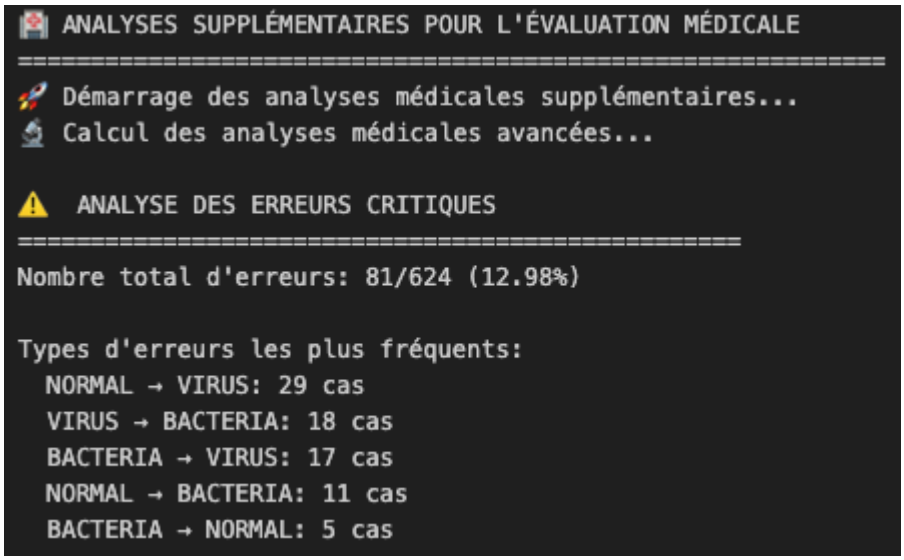
Cela permet au modèle de **ne pas favoriser uniquement la classe majoritaire**.

5.8. Conclusion

Ce projet démontre la pertinence d'un CNN simple mais bien régularisé pour la classification multiclasse de radiographies pulmonaires. La validation croisée montre

une **bonne généralisation**, et la gestion du déséquilibre contribue à une meilleure équité entre les classes.





L'approche par KNN se montre insuffisante, ce qui valide le choix de l'apprentissage profond.

5.9. Améliorations possibles

- Augmenter la **résolution des images** pour capter davantage de détails
- Explorer d'autres techniques d'augmentation (ex. bruit, éclairage)
- Tester des méthodes d'ensemble (voting, stacking)

6. Rapport sur le Modèle InceptionV3

6.1. Introduction

6.1.1Objectif du projet

Ce projet vise à développer un modèle de classification d'images médicales pour détecter trois types d'anomalies pulmonaires : normales, d'origine bactérienne ou virale à partir de radiographies thoraciques. Le jeu de données utilisé contient trois classes et l'objectif est d'entraîner un modèle capable de distinguer efficacement entre ces catégories.

6.1.2 Pourquoi InceptionV3

Le choix d'InceptionV3 s'explique par sa capacité à capturer différentes échelles d'information visuelle grâce à ses blocs multi-filtres. De plus, il est pré-entrainé sur ImageNet, ce qui permet d'exploiter le transfert learning pour gagner en précision et réduire les coûts d'entraînement. Ce modèle est reconnu pour sa rapidité et sa précision même sur des machines a vec des ressources limitées.

6.2. Compréhension de l'architecture InceptionV3

6.2.1 Fonctionnement global

InceptionV3 est un réseau de neurones convolutifs (CNN) profond conçu pour capturer les détails à différentes échelles dans une image. Il utilise des "Inception blocks" combinant plusieurs convolutions (1x1, 3x3, 5x5) et des poolings en parallèle, combinés pour produire une représentation détaillée.

6.2.2 Intérêt par rapport à d'autres CNN

Par rapport à des modèles classiques (comme VGG16 ou ResNet), InceptionV3 est plus efficace en termes de performances/précision grâce à sa structure parallèle et à sa profondeur optimisée (factorisation de convolution). Il présente également un meilleur compromis entre précision et coût.

3. Méthodologie d'entraînement

Prétraitement des données

Les images sont redimensionnées à **299x299 pixels** (taille standard pour InceptionV3). On applique la fonction **preprocess_input** pour normaliser les pixels comme prévu pour le modèle.

Augmentation des données avec ImageDataGenerator

L'entraînement s'appuie sur un générateur d'images (via I**mageDataGenerator**) qui applique des transformations aléatoires aux images du jeu d'entraînement. Cette technique, appelée data augmentation, permet de **simuler un jeu de données** plus varié à partir d'un ensemble initial fixe.
Parmi les transformations courantes utilisées : rotations, zooms, translations, retournements horizontaux, etc.

Cette approche a deux avantages majeurs :

- **Réduire le surapprentissage** en exposant le modèle à des variations plus larges de la même image.
- **Renforcer la généralisation** du modèle sur de nouvelles données en le rendant plus robuste aux petites variations visuelles.

En revanche, le générateur de validation n'applique aucune transformation, afin de garantir une évaluation fiable et représentative des performances réelles du modèle.

Modèle utilisé (transfert learning + fine-tuning)

Afin d'adapter ce modèle à notre propre tâche à trois classes, nous avons retiré sa dernière couche de classification (include_top=False) et ajouté une architecture personnalisée :

- **GlobalAveragePooling2D** : cette couche permet de réduire la dimension des features map tout en conservant l'essentiel de l'information. Elle transforme les sorties du modèle en un vecteur plus compact.
- **Dense(1024, activation='relu')** : une couche dense intermédiaire avec la fonction d'activation ReLU, qui permet au réseau d'apprendre des représentations non linéaires plus complexes.
- **Dropout(0.5)** : cette couche désactive aléatoirement 50% des neurones pendant l'entraînement. Cela aide à éviter le surapprentissage (overfitting) en forçant le réseau à ne pas dépendre de quelques neurones spécifiques.
- **Dense(3, activation='softmax')** : couche de sortie avec trois neurones correspondant aux trois classes, et une activation softmax pour donner des probabilités de classification.

Choix des hyperparamètres

Le choix des hyperparamètres est crucial pour assurer une convergence rapide et stable du modèle, tout en maximisant la capacité à généraliser sur des données non vues.
Voici les choix effectués :

- **Optimiseur : Adam**
  Adam est un optimiseur très utilisé car il ajuste automatiquement le taux d'apprentissage pour chaque paramètre. Cela le rend efficace pour les réseaux profonds, notamment sur des données bruyantes ou déséquilibrées. Il permet un entraînement rapide sans avoir besoin de beaucoup de réglages manuels.
- **Fonction de perte : Categorical Crossentropy**
  Cette fonction est utilisée pour les classifications à plusieurs classes où les sorties sont représentées sous forme de vecteurs one-hot. Elle permet de mesurer l'écart entre ce que le modèle prévoit et la réalité, ce qui est essentiel pour bien entraîner le modèle sur nos trois classes.
- **Métriques :**
  Ces métriques permettent d'évaluer le modèle sous différents angles :
  - **Accuracy** donne une idée globale de la performance.
  - **Precision** est utile pour éviter les faux positifs, notamment pour les classes rares.
  - **Recall** est crucial pour s'assurer que peu de cas positifs sont manqués, ce qui est important dans un contexte médical.
- **Batch size** : 32 :
  Un compromis entre vitesse d'entraînement et stabilité. Des tailles plus petites peuvent rendre le gradient plus bruyant mais facilitent la généralisation, tandis que des tailles trop grandes peuvent consommer trop de mémoire et conduire à un surapprentissage.
- **EarlyStopping** sur val_loss avec patience de 3 :
  Cette stratégie permet d'arrêter l'entraînement si la performance en validation n'évolue plus pendant 3 époques consécutives. Elle prévient l'overfitting et optimise le temps d'entraînement en évitant des époques inutiles

Entraînement initial

Le modèle est entraîné pendant 10 époques avec les couches gelées. Une sauvegarde du meilleur modèle est réalisée via **ModelCheckpoint**.

4. Itérations et améliorations apportées

Analyse des performances de base

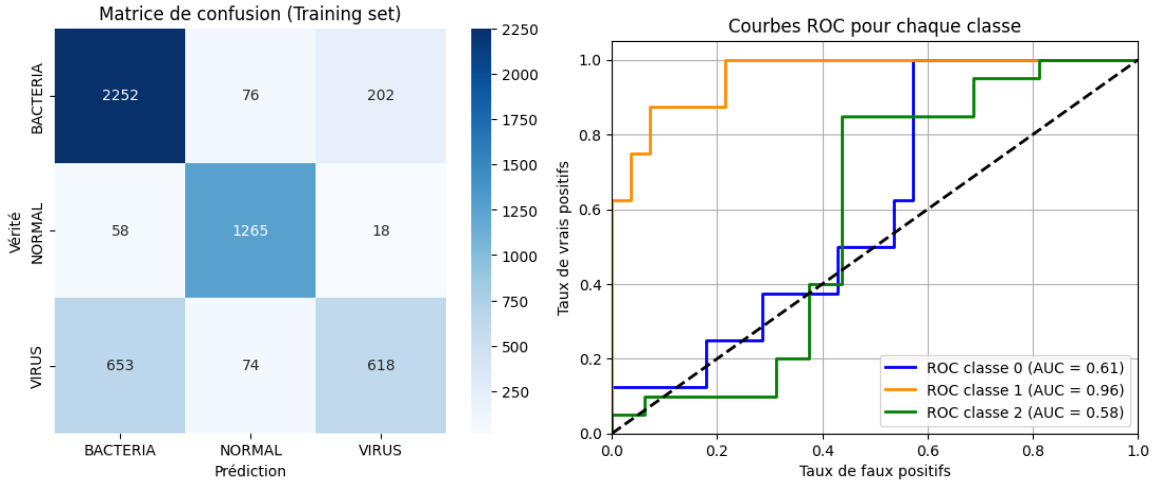
Les résultats montrent une précision correcte mais qui nécessiterait une marge d'amélioration sur les rappels et f1-score, surtout pour les classes VIRUS et BACTERIA.

Changements testés

Afin de remédier à ces limites, plusieurs ajustements ont été testés :

- **Fine-tuning** des dernières couches du modèle de base : pour permettre une adaptation plus spécifique aux caractéristiques médicales des images.
- Ajout de **Dropout** : pour limiter l'overfitting en réduisant la dépendance du modèle à certaines activations internes.
- Utilisation de **courbes ROC/AUC** et **matrices de confusion** : afin de mieux visualiser la capacité de discrimination du modèle, notamment sur les classes déséquilibrées.
- Analyse par classe via **classification_report** : afin d'obtenir une vue détaillée des performances (precision, recall, f1-score) pour chaque classe, ce qui guide les actions correctives à entreprendre (ex : collecte de plus de données, pondération de classes).

Comparaison des résultats



Après un premier entraînement du modèle, les performances initiales étaient relativement modestes, en particulier sur les données de validation. Malgré une précision d'entraînement atteignant 78 % à l'époque 8, la précision sur la validation restait inférieure à 42 %, avec un rappel autour de 38 %. Cela suggérait un phénomène de surapprentissage, où le modèle apprenait bien les données d'entraînement mais peinait à généraliser.

Fine-tuning et ajustements

Afin d'améliorer la capacité du modèle à généraliser, plusieurs techniques de fine-tuning ont été mises en place :

- Réduction adaptative du taux d'apprentissage via **ReduceLROnPlateau** pour affiner les dernières itérations.
- **EarlyStopping** pour éviter le surentraînement inutile.
- **ModelCheckpoint** pour sauvegarder le meilleur modèle basé sur la validation.
- Enrichissement des **métriques suivies** (accuracy, precision, recall, et AUC).

Ces choix ont permis d'optimiser le compromis biais-variance tout en adaptant dynamiquement l'apprentissage selon la qualité de la validation.

Résultats après fine-tuning

L'évaluation finale du modèle a été réalisée à l'aide du **rapport de classification,** qui donne une vue détaillée de la qualité de prédiction pour chaque classe.

Exemple de résultat sur les données de validation :

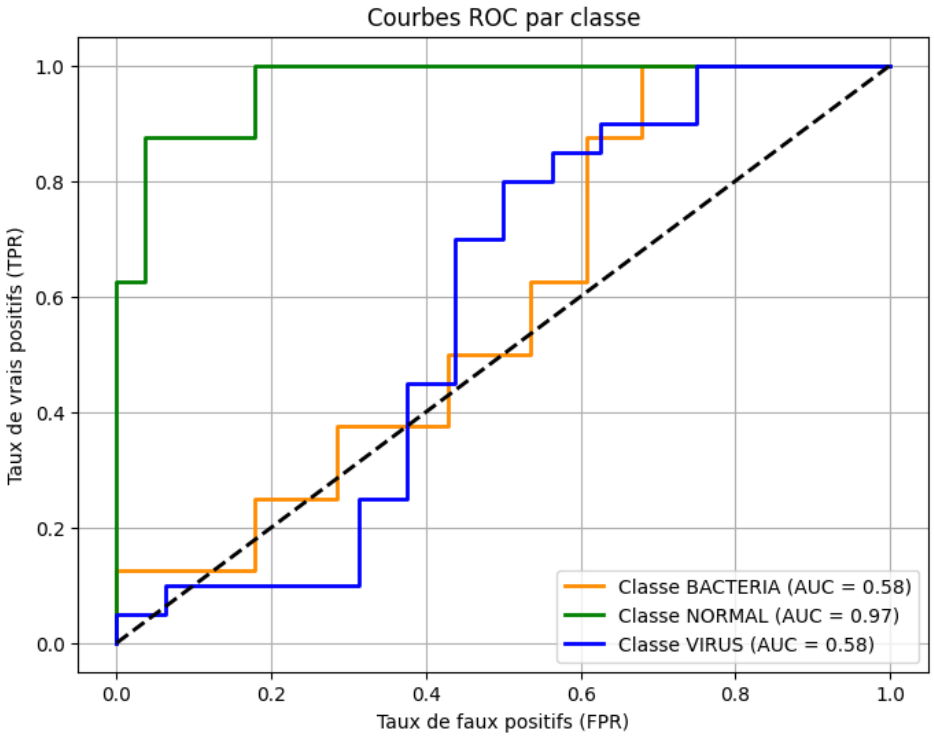
	precision	recall	f1-score	support
0	0.21	0.50	0.30	8
1	0.78	0.88	0.82	8
2	0.38	0.15	0.21	20

Très bonne détection des cas normaux : F1-score élevé à 0.82, précision à 78 % et rappel à 88 %.

Baisse de performance sur les classes BACTERIA et VIRUS :

- BACTERIA est reconnue avec un rappel de 50 % mais une faible précision (21 %).
- VIRUS a une précision faible (38 %) et un rappel très faible (15 %), ce qui indique que beaucoup de cas sont manqués.

Les courbes ROC ont également été tracées pour évaluer la capacité discriminante du modèle pour chaque classe. Voici les AUC obtenues :



Bien que les performances globales du modèle soient acceptables, les résultats obtenus révèlent une grande disparité entre les classes

On observe ainsi :

- Une excellente performance pour la classe "Normal" (AUC de 0.96), ce qui indique que le modèle est particulièrement efficace pour détecter les cas sains.
- En revanche, les performances sont nettement plus faibles pour les classes "Bacteria" (0.58) et "Virus" (0.58), suggérant une confusion fréquente entre ces deux types d'infections.
- Cette difficulté peut s'expliquer par la similarité visuelle des symptômes radiographiques entre les infections bactériennes et virales, et/ou un déséquilibre des données entre ces deux classes.

Même si le modèle arrive à différencier les cas normaux des cas malades, cela ne suffit pas pour poser un vrai diagnostic médical.

6. Conclusion

Le modèle InceptionV3, affiné par fine-tuning et enrichi par des techniques d'augmentation de données et de régularisation, montre une capacité notable à identifier les cas normaux avec un haut niveau de fiabilité. Toutefois, ses performances restent limitées sur les classes VIRUS et BACTERIA, probablement en raison d'un

déséquilibre du jeu de données et de la similarité visuelle entre ces deux types d'infections.

Conclusion Générale – Comparaison des Modèles

Au cours de ce projet, quatre approches ont été testées pour la classification multiclasse de radiographies pulmonaires (NORMAL, BACTERIA, VIRUS). Chaque modèle a été évalué à travers une méthodologie rigoureuse, incluant le prétraitement des données, la validation croisée, l'augmentation de données et l'analyse détaillée des métriques. Voici une synthèse comparative :

1. KNN (baseline simple)

- **Performance moyenne** : ~60 % de précision, confusion élevée entre classes.
- **Avantages** : très facile à implémenter.
- **Inconvénients** : ne tient pas compte des caractéristiques visuelles complexes, très sensible au bruit, ne gère pas bien la variation d'échelle ou d'orientation.
- **Conclusion** : modèle non adapté à une tâche d'analyse médicale d'images. Servait uniquement de point de départ.

2. CNN personnalisé

- **Performance moyenne** : accuracy de **82–85 %**, bonne stabilité.
- **Avantages** : architecture simple et efficace, bonne généralisation avec peu de surapprentissage, très bon rappel pour les cas normaux.
- **Limites** : confusion persistante entre les classes VIRUS et BACTERIA, AUC plus faible pour ces dernières.
- **Conclusion** : bon compromis entre complexité et performance, **suffisant pour un prototype**, mais limité pour un usage clinique exigeant.

3. InceptionV3 (transfert learning + fine-tuning)

- **Performance moyenne** : accuracy max. **78 % en entraînement**, mais **précision <45 % en validation pour certaines classes**.
- **Avantages** : bon sur les cas "NORMAL" (AUC >0.95), architecture puissante.

- **Limites** : **fort surapprentissage**, très mauvais rappel pour VIRUS (~15 %), faible généralisation, instabilité des scores sur classes déséquilibrées.
- **Conclusion** : malgré son potentiel, le modèle InceptionV3 **ne surpasse pas DenseNet121** ni le CNN personnalisé sur ce dataset. Il souffre d'un déséquilibre marqué entre classes.

4. DenseNet121 (transfert learning + fine-tuning + data aug)

- **Performance moyenne** : accuracy **77–79 %**, AUC global > 0.90.
- **Points forts** :
    - Très bonne stabilité sur plusieurs folds (validation croisée).
    - Capacité à bien distinguer "NORMAL" et "BACTERIA" (précision/rappel élevés).
    - **Fine-tuning + augmentation** ont permis une amélioration constante et fiable.
- **Limites** : la classe "VIRUS" reste difficile à discriminer (comme pour les autres modèles), mais l'erreur est moins marquée que chez InceptionV3.
- **Conclusion** : modèle le plus **équilibré et robuste**, avec des résultats solides, peu de surapprentissage, et **meilleure généralisation** sur l'ensemble du jeu de données.

7. Annexe

7.1. Méthode de calcul

Valeur	Méthode
Précision	TP / (TP+FP)
Rappel	TP / (TP+FN)
Score F1	2 x (Précision * Rappel) / (Précision * Rappel)

AUC	Aire sous la courbe ROC tracée à partir de TPR = TP / (TP + FN) et FPR = FP / (FP + TN)

TP => Nombre de vrais Positifs
FP => Nombre de faux Positifs
TN => Nombre de vrais Négatifs
FN => Nombre de faux Négatifs
TPR => Taux vrais Positifs
FNR => Taux faux Négatifs